RAFFLES INSTITUTION
H2 Computing (9569)

2025 Year 5

Chapter S1E: Queue

CONTENT

No.	Topic	Syllabus Guide
1	Introduction to Queue	1.3.3
2	Queue Operations	
3	Queue Overflow and Underflow	
4	Usage of Queue	
5	Linear Queue 5.1 Enqueue 5.2 Dequeue	
6	Circular Queue 6.1 Enqueue 6.2 Dequeue	
7	Comparison of Queue Implementation	
8	Tutorial	
9	Programming Exercise	

1. INTRODUCTION TO QUEUE

A **queue** is a collection of sequential data in which new data is added to the "end" of the queue while data can only be removed from the "front". A queue is a "FIFO" data structure (i.e. **First In First Out**). The first entry placed in the queue will be the first entry removed from the queue, and the last entry placed in the queue will be the last entry removed from the queue.

In a normal queue where you wait in line to buy tickets at a counter, you join the line at the end, and as people in front of you get served, they move forward and eventually it is your turn. This idea of people waiting in a line and being served in a specific order represents a queue data structure.

Unlike a normal ticketing queue, data in a queue data structure is not allowed to ***move along***. Instead, each data ***stays put*** in its storage location until its turn comes.

Two pointers, the front pointer indicating the "front" and the rear pointer indicating the "end" of the queue are used.

An example of a logical view of a queue with 5 items:

LOCATION	101	102	103	104	105	106	107	108	109	110
CONTENT	20	4	8	12	11					

Front Pointer → 101 ; Rear Pointer → 105

2. QUEUE OPERATIONS

The following operations can be used to implement a queue.

- `enqueue(item)` Add a new item to the rear of the queue
- `dequeue()` Remove the front item from the queue and return it
- `isEmpty()` Test to see whether queue is empty
- `isFull()` Test to see whether queue is full
- `size()` Return the number of items in the queue.

3. QUEUE OVERFLOW AND UNDERFLOW

Overflow occurs when an attempt is made to add data to a queue when all available locations are occupied.
Underflow occurs when an attempt is made to remove data from an empty queue.

4. USAGE OF QUEUE

4.1 Buffer

Queues are commonly used in the buffer area of the main storage to handle data for input/output operations. The processor can then handle input and output operations efficiently without being delayed by the slower speeds of peripheral devices. The queue ensures that data is processed in a fair and ordered manner, maintaining the integrity of the input/output operations.

When you type on a keyboard, the keystrokes are stored in an input buffer. The keyboard controller interrupts the processor to signal that data is available. The processor then retrieves the data from the front of the input buffer and processes it. This ensures that the keystrokes are processed in the order they were received.

When you send data to an output device, such as a printer, it is stored in an output buffer. The processor retrieves the data from the front of the output buffer and sends it to the printer. The printer can process the data at its own pace, while the processor can continue with other tasks. The output buffer ensures that data is sent to the printer in the order it was requested.

4.2 Spooling

Output waiting to be printed is commonly stored in a queue on disk. In a networked environment where multiple computers share a single printer, several print jobs can be sent simultaneously. By storing the output in a disk-based queue, a first come, first served approach is established, ensuring that the output is printed as soon as the printer becomes available. The disk-based queue provides persistent storage, allowing print jobs to be retained even in the event of system shutdown or crashes. Spooling, in combination with a disk-based queue, enables efficient management and concurrent processing of print jobs, enhancing the printing experience in networked environments.

4.3 Queue-based simulations

Queues are also prevalent in computer simulations that aim to understand and minimize queuing time in real-life systems. By incorporating random numbers and statistical data, simulations can model various queuing scenarios, such as airport plane landing and takeoff queues, petrol pump queues, or customers arriving at cashier/teller counters.

By employing queues in these simulations, researchers gain insights into the effects of queuing and explore strategies to optimize queuing times. This knowledge can lead to improved operational efficiency in real-world scenarios, benefiting areas like transportation, service industries, and more.

5 LINEAR QUEUE

A linear queue can be implemented using an array with the use of 3 variables:

- **`front`**: indicate front of the queue
- **`rear`**: indicate rear of the queue
- **`size`**: indicate number of elements in queue

Assume the array is called **`queue`**.

Example: Initialise empty queue

Index	0	1	2	3	4	5	6	7	8	9
Content										

- **`front`** = 0 **`rear`** = –1 **`size`** = 0

Example: Queue with 8 items

Index	0	1	2	3	4	5	6	7	8	9
Content	20	4	8	12	11	88	32	56		

- **`front`** = 0 **`rear`** = 7 **`size`** = 8

5.1 Insert into linear queue (enqueue)

```
PROCEDURE Enqueue(item)
    IF rear = maxIndex THEN write ("Queue Overflow")
    ELSE
        rear = rear + 1
        queue[rear] = item
        size = size + 1
    ENDIF
ENDPROCEDURE
```

Example: Enqueue(99)

Index	0	1	2	3	4	5	6	7	8	9
Content	20	4	8	12	11	88	32	56	99	

- **front** = 0 **rear** = 8 **size** = 9

The item 99 is added to rear of queue. Rear pointer is incremented. Size is increased by 1.

5.2 Delete from linear queue (dequeue)

```
FUNCTION Dequeue
    IF size = 0 THEN write ("Queue Underflow")
    ELSE
        deleted_item = queue[front]
        front = front + 1
        size = size - 1
        RETURN deleted_item
    ENDIF
ENDFUNCTION
```

Example: Dequeue

Index	0	1	2	3	4	5	6	7	8	9
Content	20	4	8	12	11	88	32	56	99	

- **front** = 1 **rear** = 8 **size** = 8

The item 20 is removed from front of queue. Note that front pointer is incremented but the data in the array does not changed. Size is decreased by 1.

Example: Enqueue(77)

Index	0	1	2	3	4	5	6	7	8	9
Content	20	4	8	12	11	88	32	56	99	77

- **front** = 1 **rear** = 9 **size** = 9

The item 77 is added to rear of queue. Rear pointer is incremented. Size is increased by 1.

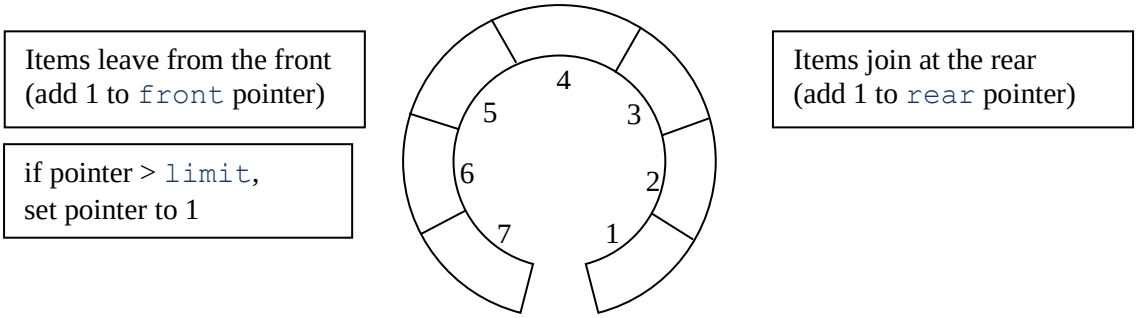
At this point, all the locations in the linear queue are occupied. No additional items can be added into this linear queue. Even though the first item (in index 0) was already removed, memory is not reused.

One solution to reuse the memory is to shift each item one to the left, every time an item is removed from the front. However, this will potentially slow down the processing if there are many items in the queue as it takes time to shift the items.

6 CIRCULAR QUEUE

A better solution is to use a data structure called **circular queue**. A fixed area of store for the queue is allocated and the rear of the queue "wrap around" to the start of the area.

The first item is held in a location which is logically next to the storage location for the last item of the queue.



Initialising the queue:

Note that the rear of the queue is initialised to limit, because in the circular representation of the queue, this element immediately precedes the first element. The rear pointer is moved forward just before a new element joins the queue, as shown in the following pseudocode.

6.1 Insert into circular queue (enqueue):

```
PROCEDURE enqueue(item)
    IF size = limit THEN write ('Queue Overflow') //check for overflow
    ELSE
        IF rear = limit THEN rear = 1          //advance the rear pointer
        ELSE rear = rear + 1
        ENDIF
        queue[rear] = item                     //store item into rear
        size = size + 1                        //increment size
    ENDIF
ENDPROCEDURE
```

6.2 Delete from circular queue (dequeue):

```
FUNCTION dequeue
    IF size = 0 THEN write ('Queue Underflow') //check for underflow
    ELSE
        deleted_item = queue[front]    //store front item into deleted_item
        size = size - 1                //decrement size
        IF front = limit THEN front = 1
        ELSE front = front + 1         //advance the front pointer
        ENDIF
        RETURN deleted_item
    ENDIF
ENDFUNCTION
```

7. COMPARISON OF QUEUE IMPLEMENTATION

There are several ways to implement a queue. Each method has its own advantages and disadvantages in terms of memory usage, efficiency, and complexity.

- **Linear Queue (Array-based)**
  - Fixed-size structure where elements are added to the rear and removed from the front.
  - Memory is not reused when elements are removed from the front unless the entire array is shifted (which is inefficient).

- **Circular Queue (Array-based)**
  - Enhances the linear queue by reusing memory locations at the front of the array.
  - When the rear reaches the end, it wraps around to the beginning.
  - Slightly more complex pointer handling, but no shifting is needed.

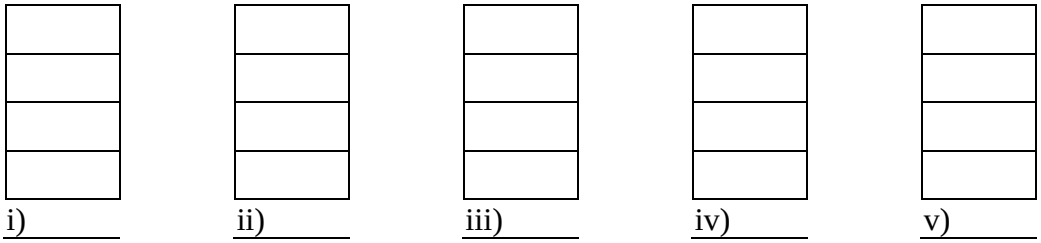
- **Linked List Queue**
  - A queue can also be implemented using a linked list where each node contains data and a pointer to the next node.
  - New nodes are added at the rear, nodes are removed from the front.
  - Both pointers (front and rear) are updated to point to the appropriate nodes during enqueue and dequeue operations.

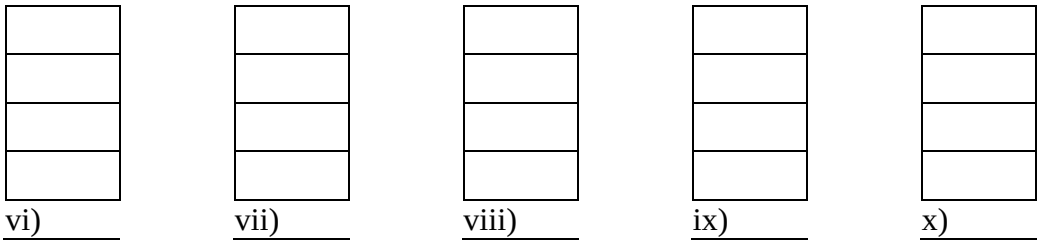
Feature	Linear Queue	Circular Queue	Linked List Queue
Memory Usage	Fixed-size; unused spaces are not reused	Reuses memory efficiently by wrapping around	Dynamic; grows as needed
Overflow Condition	When `rear` reaches the end of array	When `size` = `limit`	No overflow (unless memory runs out)
Reusability	No (unless shifting)	Yes	Yes (nodes created/ deleted dynamically)
Pointer Movement	`rear` and `front` only move forward	`rear` and `front` wrap to beginning	`rear` and `front` move to newly created or deleted nodes dynamically
Implementation Complexity	Easier to implement	Slightly more complex logic (wrap-around)	Higher complexity due to pointers needed
Performance	Fast access using index; slower when shifting elements (if done)	No need for shifting elements, hence more efficient	If `rear` and `front` pointers are maintained, traversal is not needed. If not, pointer traversal can be slow.

8. TUTORIAL

1. A linear queue with 4 locations is set up. Insert the Rear Pointer, Front Pointer, the content of the queue and error messages (if any) in the diagrams below after the following steps are performed:

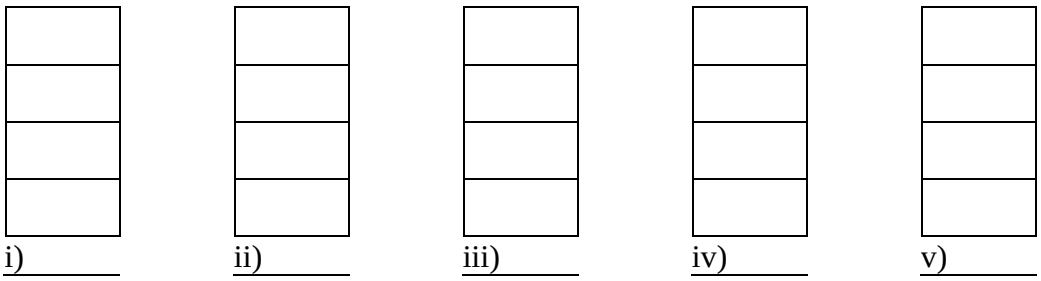
i) Set up the queue	ii) Insert data [50]
iii) Delete	iv) Delete
v) Insert data [35]	vi) Insert data [40]
vii) Insert data [80]	viii) Insert data [10]
ix) Delete	x) Insert data [03]

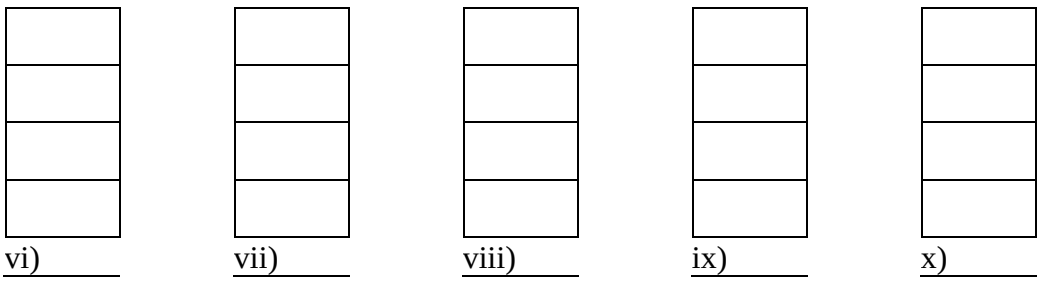




2. A circular queue with 4 locations is set up. Insert the Rear Pointer, Front Pointer, the content of the queue and error messages (if any) in the diagrams below after the following steps are performed:

i) Set up the queue	ii) Insert data [50]
iii) Delete	iv) Delete
v) Insert data [35]	vi) Insert data [40]
vii) Insert data [80]	viii) Insert data [10]
ix) Delete	x) Insert data [03]





3. Complete the table by showing the queue contents and the return value of the queue operation. The queue is named q. The method enqueue adds an element to the queue while dequeue removes an element from the queue and returns the value of the element removed.

Queue operation	Queue contents	Return value
`q.isEmpty()`	`[]`	`True`
`q.enqueue("Ben")`	`["Ben"]`	`None`
`q.enqueue("Sam")`	`["Ben","Sam"]`	`None`
`q.enqueue("Tan")`		
`q.isFull()`		
`q.isEmpty()`	`["Ben","Sam","Tan"]`	
`q.dequeue()`		
`q.enqueue("Ron")`		
`q.dequeue()`		
`q.dequeue()`		
`q.isEmpty()`		

9. PROGRAMMING EXERCISE

1. Implement a circular queue ADT using array with the following properties and methods:

Class: `Queue`		
Properties		
Identifier	Data Type	Description
`Data`	`ARRAY[20] OF STRING`	Array to hold the queue elements. The array index starts at 1.
`Front`	`INTEGER`	Index position of the first element of the queue in the Data array.
`Rear`	`INTEGER`	Index position of the rear element of the queue in the Data array
`Limit`	`INTEGER`	The maximum number of locations of the queue.
`Size`	`INTEGER`	The number of elements in the queue
Methods		
`Constructor`	`PROCEDURE`	Set up an empty queue and initialise values for its attributes.
`IsEmpty`	`FUNCTION RETURNS BOOLEAN`	Indicates whether any elements are stored in queue or not.
`IsFull`	`FUNCTION RETURNS BOOLEAN`	Indicates whether queue is full or not.
`Enqueue`	`PROCEDURE`	Inserts data onto queue.
`Dequeue`	`FUNCTION`	Removes and returns the front element of the queue.
`Peek`	`FUNCTION`	Returns the front element from the queue without removing it.
`GetSize`	`FUNCTION`	Returns the number of elements stored in queue.
`Display`	`PROCEDURE`	Displays the content of the queue with front and rear pointers clearly indicated.

Test your circular queue and show that it works.